

CA-MCPQ: A Context-Aware Post-Quantum Protocol for AI Agent Integrity and Security

Seyoung Yoon^[0009-0008-3254-9256], Hyunji Kim^[0000-0001-9828-3894], and
Hwajeong Seo^[0000-0003-0069-9061]

Hansung University, South Korea
{sebbang99,khj1594012,hwajeong84}@gmail.com

Abstract. We propose CA-MCPQ, a context-aware post-quantum-secure extension of the Model Context Protocol (MCP). Unlike standard MCP, which leaves authentication, encryption, and authorization optional or implementation-specific, CA-MCPQ elevates them to mandatory protocol-level mechanisms. The design incorporates post-quantum mutual authentication, KEM-derived session keys, and authenticated sequencing to ensure session integrity and prevent replay. Role-based access control is enforced, while token-based authentication is eliminated entirely. AI dynamically infers the required security tier from contextual information and negotiates compatible PQC algorithms; each response returns a reliability score that quantifies alignment with the requested security level.

This architecture addresses critical vulnerabilities of MCP—including token misuse, session hijacking, impersonation, and quantum attack—while preserving interoperability. Notably, our evaluation shows that the cryptographic and transport overheads are negligible compared to model computation, confirming that strong post-quantum assurances can be achieved without degrading system performance. Overall, CA-MCPQ provides a practical path toward secure-by-design AI agent ecosystems and lays the groundwork for future extensions such as agent-agent secure coordination.

Keywords: Model Context Protocol (MCP) · Post-Quantum Cryptography (PQC) · AI Agent · Trustworthy AI

1 Introduction

Large language models (LLMs) extend strong reasoning capabilities but remain confined to the knowledge encoded during training, limiting their ability to answer queries requiring real-time or domain-specific information. To overcome this limitation, AI agents extend LLMs by accessing and integrating external information sources such as databases, web services, and enterprise APIs. The Model Context Protocol (MCP) was recently introduced as an open standard to support these agent-tool interactions, providing a unified framework for capability

discovery and invocation. However, despite its rapid adoption, MCP still suffers from critical security gaps, including the absence of standardized mechanisms for authentication, authorization, and session integrity.

Beyond these immediate risks, the cryptographic underpinnings of MCP face longer-term threats. Advances in quantum computing threaten the viability of widely deployed public-key algorithms, creating the prospect that encrypted traffic captured today could be decrypted in the future. This “harvest-now, decrypt-later” scenario undermines long-term confidentiality and authenticity, raising the need for post-quantum cryptographic (PQC) protections in agent–tool protocols.

In this work, we advance MCP toward a secure-by-design architecture, which we call Model Context Protocol Secure (MCPS). Our approach emphasizes context-aware security, enabling the agent itself to evaluate the sensitivity of prompts or data and autonomously enforce appropriate protections. By combining protocol-level security primitives with adaptive PQC integration, the system moves beyond static safeguards toward a model in which agents dynamically calibrate authentication, encryption, and authorization to the demands of each interaction. This perspective lays the foundation for agentic ecosystems that remain both interoperable and resilient under emerging quantum-era threats.

To the best of our knowledge, this is the first protocol-level integration of context-aware post-quantum cryptography into MCP.

The remainder of this paper is organized as follows. Section 2 provides background and related works. Section 3 presents the system design and implementation. Section 4 evaluates the performance and security of the proposed approach. Finally, Section 5 concludes with a discussion of limitations and directions for future work.

1.1 Contribution

This paper presents a post-quantum secure extension of the MCP, which integrates dynamic security mechanisms into the agent–tool interaction layer. Our contributions can be summarized as follows:

- **Context-aware dynamic security.** We present a *Context-Aware Cryptographic Adaptation* mechanism that tightly integrates the agent’s decision-making process with post-quantum cryptographic controls. The agent autonomously evaluates contextual attributes of user prompts or document classifications, infers the required security level (i.e., NIST post-quantum security [1,2]), and dynamically selects the appropriate PQC algorithm suite to re-establish the session. This approach balances performance with data sensitivity and establishes a new paradigm in which agents enforce security policies adaptively rather than relying solely on static role assignments.
- **Reliability scoring for PQC-based responses.** We propose a reliability scoring mechanism that quantifies the trustworthiness of each response according to the alignment between the required security level and the PQC algorithms supported by the server. When the server fully satisfies the inferred requirements, the response is assigned the maximum score; when support is

partial or missing, the score is reduced accordingly. This scoring not only enables the agent to assess and adapt to heterogeneous environments, but also enhances the overall dependability of the system by providing a measurable indicator of response integrity, thereby fostering more trustworthy AI-tool interactions.

- **Mutual authentication with post-quantum primitives.** We extend MCP with a protocol-level mutual authentication mechanism in which the server proves its authenticity using post-quantum digital signatures, while the client is verified through a PQC-based challenge-response procedure. This design ensures that both parties are cryptographically authenticated, providing strong resistance against impersonation even in the presence of quantum-capable adversaries.
- **MCP protocol extension with integrated security.** We design MCPS as a drop-in extension of MCP, preserving JSON-RPC compatibility while incorporating protocol-level tools for mutual authentication, algorithm negotiation, session key establishment, and role-based access control (RBAC). By embedding these mechanisms at the protocol layer, security enforcement is no longer delegated to ad-hoc application design.

2 Background

2.1 AI Agent

An AI agent can be defined as an autonomous system that perceives inputs, reasons over goals and contextual information, and executes actions upon an external environment [3]. In contemporary implementations, such agents are typically realized by coupling a large language model with a control loop that orchestrates task decomposition, tool selection, execution, and verification. This loop is generally structured into three stages: planning, memory, and action. Planning transforms an open-ended request into a sequence of well-defined operations; memory preserves conversational state in both short-term and long-term forms; and actions invoke external tools, services, or code, with outputs fed back into the model for iterative reflection and error recovery.

In contrast to standalone language models, which respond exclusively from pre-trained knowledge, agents are capable of integrating external information sources. They can query enterprise databases, invoke web APIs, execute code, and coordinate multi-step workflows, thereby enabling tasks that require up-to-date data, system integration, or compliance with organizational policies. Given this extended operational scope, agents must adhere to safety and governance requirements. Typical enforcement mechanisms include policy-guided planning, schema validation of inputs and outputs, and systematic logging for auditability. These controls become substantially more reliable when the interface between agents and external tools is standardized, thereby motivating protocol-level support for agent-tool interaction [3,4].

2.2 Model Context Protocol

MCP is an open standard designed to unify how AI agents use external tools and services. It defines a clear client–server model: the agent operates as a client that discovers and invokes tools, while servers expose these tools with precise schemas that specify inputs, outputs, and possible errors [5,6]. A typical workflow begins with the agent requesting a list of available capabilities, continues with one or more tool invocations, and concludes with a structured response that can be validated and reused (see Figure 1). While offering substantial benefits for AI systems, MCP also exposes several critical security vulnerabilities.

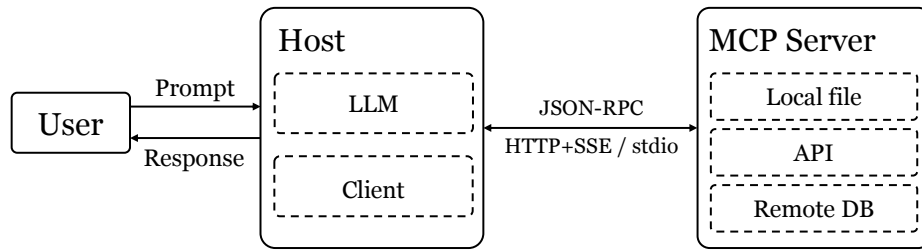


Fig. 1: Overview of MCP workflow.

MCP’s primary goal is to make tool usage predictable and interoperable across different systems. By relying on typed schemas, it ensures that model-generated outputs match the formats expected by tools, reducing errors and simplifying integration. Standardized error handling allows agents to recover gracefully or choose fallback options when a tool call fails. Because MCP is transport-agnostic, it can be deployed over common channels such as HTTP or standard I/O, which makes it portable and adaptable to diverse environments. Through these features, MCP provides a functional foundation for agents to interact with tools in a reliable, consistent, and standardized manner [5].

2.3 Security Properties and Vulnerabilities in AI Agent and MCP

The security properties and vulnerabilities of AI agent protocols such as MCP depend both on their protocol design and on the cryptographic primitives they employ (see Table 1). We categorize the issues into protocol-level vulnerabilities and quantum vulnerabilities, reflecting the distinct but complementary threat vectors relevant to MCP deployments.

Protocol-Level Vulnerabilities. MCP was introduced to standardize how agents interact with external tools, but current implementations reveal critical gaps. Notable risks include the absence of mandatory mutual authentication, reliance on optional transport-layer encryption, and lack of standardized authorization frameworks. These deficiencies can result in impersonation, replay, or over-privileged

Table 1: Security vulnerabilities in MCP and AI agent protocols.

Category	Key Risks
Authentication / Authorization	Lack of mutual authentication, insufficient access control
Session Management	Session hijacking, replay attacks
Token Handling	Token passthrough, stolen OAuth token reuse
Protocol-Level Gaps	Optional encryption, over-privileged tool access
Quantum Threats	Shor and Grover’s algorithm

tool access, especially in enterprise settings where agents interact with sensitive resources.

Authentication and Authorization Deficiencies. Red Hat’s security analysis [7] indicates that current MCP implementations exhibit fundamental authentication weaknesses, as the specification does not enforce security principles at the protocol level and lacks mandatory mechanisms for mutual authentication. These shortcomings pose considerable risks in enterprise environments.

Recent industry assessments have similarly noted that the absence of robust authentication controls permits unauthorized access, since identity and permission checks are not systematically enforced. This deficiency constitutes a protocol-level vulnerability that cannot be sufficiently addressed through application-layer safeguards alone. Moreover, gaps in standardized authorization frameworks exacerbate this risk. Red Hat’s assessment highlights that MCP omits standardized mechanisms for fine-grained access control, thereby enabling AI agents to retrieve sensitive organizational data without adequate clearance verification [8].

Session Management Vulnerabilities. The official MCP security documentation [9] identifies session hijacking as a primary attack vector, in which adversaries who obtain session identifiers can impersonate legitimate clients. Current implementations lack mechanisms to ensure session integrity, leaving them vulnerable to such compromises. In addition, reliance on conventional session management techniques introduces susceptibility to replay attacks, whereby intercepted communications can be maliciously reused to gain unauthorized access to protected resources. These issues have been explicitly acknowledged in the specification’s best practices guidance [9].

Token-Based Attack. The same security documentation explicitly discourages token passthrough patterns, characterizing them as an anti-pattern in which servers forward client-provided tokens to downstream APIs without validating that the tokens were legitimately issued to the MCP server [9]. Despite this prohibition, insecure token handling remains prevalent in practice.

The credential theft risk is particularly severe. Pillar Security’s analysis demonstrates that if an attacker exfiltrates an OAuth token stored by an MCP server, the token can be reused to instantiate a malicious MCP server and impersonate the legitimate service [10]. This attack vector exploits MCP’s dependence

on conventional OAuth mechanisms that lack additional layers of authentication. As Red Hat also observes, the current reliance on OAuth-based authorization introduces tensions with enterprise requirements, underscoring the need for more rigorous, standardized access control models [8].

Quantum Vulnerabilities Beyond protocol design flaws, the cryptographic primitives that underpin MCP are exposed to distinct risks from quantum-capable adversaries. Public-key schemes commonly used for authentication and key agreement (e.g., RSA, ECC) would be rendered ineffective by large-scale quantum computers running Shor’s algorithm, enabling adversaries to recover long-term private keys and compromise past communications [11,12].

Symmetric primitives are also affected, albeit to a lesser extent: Grover’s algorithm provides a quadratic speedup for brute-force search, effectively halving the security margin of fixed-size keys and necessitating larger key sizes to maintain equivalent security [13]. Compounding these concerns, the post-quantum candidate landscape continues to evolve—recent cryptanalytic advances have already displaced some proposals—so transitioning to quantum-resistant schemes requires careful selection and ongoing reassessment [14].

For MCP deployments, these factors imply two operational imperatives. First, short-term confidentiality cannot be assumed if adversaries may archive traffic for future decryption (harvest-now, decrypt-later threat). Second, long-lived authentication and key-exchange mechanisms should migrate to algorithms with demonstrated resistance to quantum attacks. Accordingly, integrating post-quantum key-encapsulation and signature primitives into MCP’s handshake, session management, and authorization workflows is essential to ensure lasting integrity and confidentiality.

2.4 Protocol-Level Security Requirements for MCP

Recent security assessments conclude that MCP was not designed with a security-first approach, cautioning that without fundamental changes, each integration may act as a potential backdoor into enterprise systems [7,9]. These observations underscore the necessity of introducing explicit protocol-level security capabilities that can adapt to the sensitivity of the task and the criticality of the resources being accessed.

To effectively mitigate the identified threats, MCP must evolve to support security requirements that are both standardized and dynamically applicable:

- **Cryptographic authentication:** A standardized mechanism for mutual verification of client and server identities is essential, with the flexibility to apply stronger guarantees for high-sensitivity contexts [8].
- **Communication encryption:** Mandatory protection of sensitive exchanges should be enforced at the protocol layer, with the ability to escalate cryptographic strength when handling confidential or regulated data [9].
- **Session integrity:** Protocol-defined safeguards against session hijacking and replay attacks must be incorporated, ensuring that integrity protections scale with session criticality [9].

- **Quantum resistance:** To prevent the quantum attack, post-quantum cryptography should be integrated by design, with security tiers that align with data retention windows and threat models [9].
- **Authorization enforcement:** Standardized, fine-grained access control frameworks are needed, enabling differentiated authorization policies that reflect user roles, data classifications, and operational risk [8].

Rather than static safeguards, what is required is a protocol framework that can dynamically calibrate the level of protection—applying stronger authentication, encryption, or access controls where context demands it, while preserving interoperability and efficiency elsewhere. Establishing such adaptable requirements moves MCP toward secure-by-design implementations capable of supporting enterprise AI agent ecosystems at scale.

2.5 Post-Quantum Cryptography

Post-quantum cryptography aims to keep public-key operations secure even when large quantum computers exist. A key risk is harvest now, decrypt later. Attacker records ciphertext today and wait until decryption becomes practical. In this case, classical key exchange and signatures may fail to provide long-term confidentiality or authenticity.

Current practice favors pragmatic migration. Crypto agility is the ability to swap primitives without disrupting protocols and libraries. In practice this means negotiable algorithms, explicit versioning, safe defaults, automated rotation, and phased rollouts. Building on that foundation, hybrid handshakes combine classical and post-quantum mechanisms so either path can secure the session while deployments yield real performance and compatibility data.

NIST operates an open program for the standardization of post-quantum cryptography (PQC) and provides transition guidance, most notably in NISTIR 8547. In parallel, the U.S. national security community has aligned on the CNSA 2.0 suite, which specifies recommended algorithms and migration timelines. The UK National Cyber Security Centre (NCSC) has published a roadmap extending through 2035, including concrete parameter recommendations for common applications. In South Korea, the KpqC competition is advancing as a domestic initiative to evaluate and standardize national PQC algorithms.

Adoption efforts are already underway across major platforms. Amazon has deployed PQC within AWS services by integrating hybrid ECDH with ML-KEM, supported through AWS Libcrypto, a FIPS 140-3 validated library that first incorporated ML-KEM into its certification. Google initiated PQC testing in Chrome as early as 2016 and has employed PQC for internal communications since 2022. In May 2024, Chrome desktop enabled ML-KEM by default for TLS 1.3 and QUIC with server-side support, thereby experimentally protecting connections to services such as Cloud Console and Gmail through post-quantum key exchange.

3 CA-MCPQ

In this section, we present the design and implementation of CA-MCPQ, our context-aware post-quantum secure extension of MCP. Figure 2 presents an overview of our design, which integrates post-quantum cryptography with context-aware security level determination to enhance the MCP. The overall workflow proceeds through the following steps:

Step 1. Prompt. The user submits a prompt to the host.

Step 2. Initial Session Negotiation and Authentication. The host and MCP server perform post-quantum mutual authentication: the server proves authenticity via a PQ digital signature, and the host is validated through a PQC-based challenge-response.

Step 3. Context-Aware Security Decision. The LLM, as part of the agent system, interprets the description associated with the request and infers the required post-quantum security level.

Step 4. Session Establishment and Encryption. According to the inferred level, the host and server negotiate and establish a secure session using post-quantum key encapsulation, while recording a reliability score that reflects the match between required and supported algorithms. Tool invocations and data exchanges are encrypted and signed within the established session.

Step 5. Verification and Decryption. The host verifies signatures, decrypts the data, and checks the reliability score of response to assess trustworthiness.

Step 6. Response Delivery. Assuming an authorized user requesting data they are permitted to access, the host returns the final response to the prompt to the user. The response includes "requested data, selected KEM/SIG suite, response reliability score, rationale for the selection, requester's authorization tier".

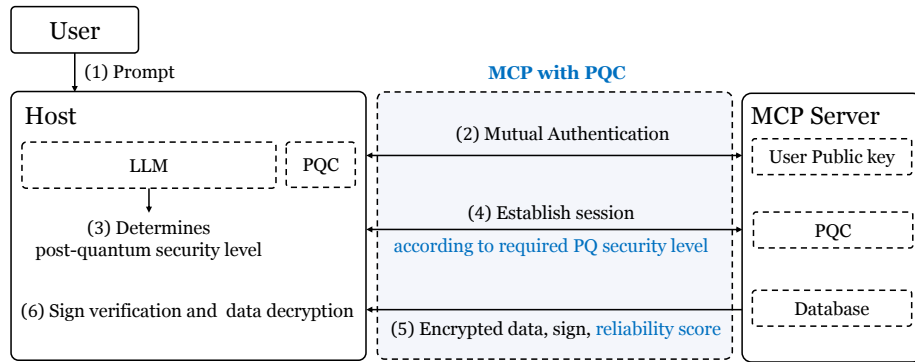


Fig. 2: Overview of our design (CA-MCPQ).

3.1 Initial Session Negotiation and Authentication

In our design, the initial session negotiation is performed using a fixed set of post-quantum algorithms. This ensures that the first exchange of keys and authentication data can be carried out with minimal complexity and consistent interoperability. After this stage, the actual KEM and DSA for the secure session are selected through a separate negotiation process¹. After the initial setup, the following session (generated for each user prompt) is established through a separate negotiation, in which the required KEM and signature algorithms are selected according to the security level inferred by the LLM.

3.2 Context-Aware Determination of Post-Quantum Security Level

After mutual authentication, LLM determines the required security level based on the user prompt and its surrounding context. Figure 3 illustrates how the required post-quantum (PQ) security level is determined in our system. Rather than relying on fixed parameters, the agent dynamically interprets task descriptions and translates them into security requirements that guide subsequent protocol negotiation.

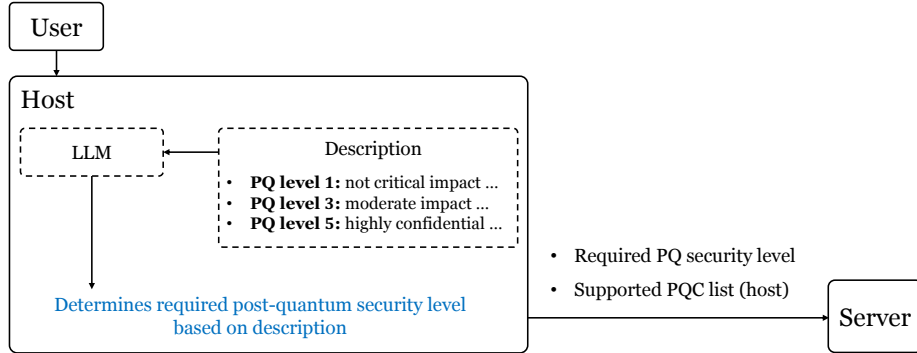


Fig. 3: Determination of post-quantum security level for required data.

LLM inference of security level. The host first constructs a task prompt that embeds relevant user context such as roles, clearance level, and accessible documents. This prompt also specifies a tool schema containing two parameters: a document identifier (`key`) and a security level (`PQLevel1`). Each security level is

¹ In production deployments, key management should rely on an HSM or cloud-based KMS, ensuring that key generation, storage, rotation, and revocation are performed in accordance with defined policies. Moreover, access control and audit logging should be integrated with the key lifecycle management framework to mitigate risks of misuse and credential replay.

described by concise criteria that distinguish low-impact, moderately sensitive, and highly confidential information. This step allows the agent to map natural-language descriptions of tasks or documents into a concrete security level without manual intervention.

Mapping to NIST post-quantum security level. The inferred `PQLevel` is then mapped to standardized NIST post-quantum tiers, ensuring interoperability across cryptographic libraries. For example, `PQLevel_1` corresponds to NIST security level 1 (corresponding to AES-128).

Preparation of supported PQC list. Once the required tier is determined, the host enumerates its locally available post-quantum algorithms and filters them to those meeting or exceeding the inferred level. The result is a candidate list of key encapsulation mechanisms and signature schemes that reflect both the security requirements and the host’s implementation preferences.

Negotiation inputs for the server. Finally, the host provides two key items to the MCP server: the required PQ security level inferred by the LLM, and the filtered list of supported PQC algorithms. These values serve as the basis for protocol-level algorithm negotiation and secure session establishment, ensuring that the level of protection is both contextually justified and cryptographically enforceable.

3.3 Session Establishment and Role-Based Access Control

This integration of PQC-based session establishment with RBAC enables the protocol to enforce fine-grained security guarantees: cryptographic protection of all exchanges, replay resistance via authenticated sequencing, and policy compliance through standardized authorization checks. Therefore, the system provides both strong cryptographic assurances and reliable governance over sensitive tool interactions in AI agent and MCP environments. The following presents the two core components of our design.

Post-Quantum Session Management Once the required security level has been determined, the host establishes a session with the server using post-quantum key encapsulation (see Figure 4). The host encapsulates a shared secret under the server’s public key (i.e., $\text{KEM.encap}(pk_s)$ in Figure 4) and transmits the resulting ciphertext (ct) to the server, which in turn applies its secret key (sk_s) to decapsulate and recover the same shared secret (i.e., ss and ss' should be the same). Both parties then derive the authenticated encryption key (i.e., session key) via key derivation function (i.e., HKDF), and this key material is bound to a fresh session context. As part of the session definition, the structure of the additional authenticated data (AAD)—covering the method name, host identifier, and sequence number—is fixed, together with the rule that all exchanges must conform to the negotiated session parameters and counters. These elements provide the basis for the encrypted communication described in the following section.

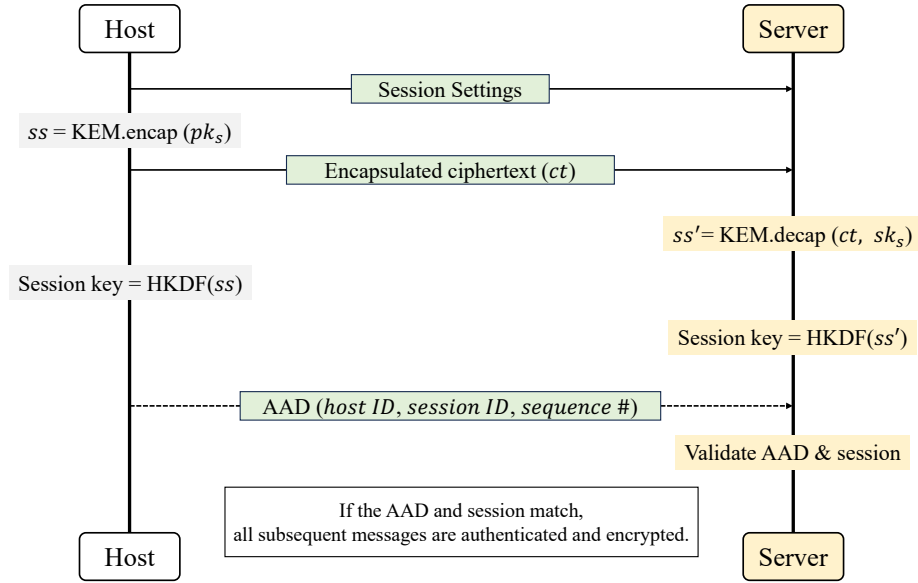


Fig. 4: Session establishment with post-quantum KEM.

Role-Based Access Control The decrypted request is then subject to role-based access control, where the client’s role and clearance are evaluated against the requested resource. If access is authorized, the server queries the underlying database and returns the result. If the request exceeds the client’s permissions, the server responds with an encrypted denial message indicating that the document cannot be accessed. In both cases, the response remains protected under the same session context, thereby ensuring confidentiality, integrity, and consistent enforcement of access policies.

3.4 Response and Reliability Scoring

Each server response is encrypted with the same AEAD key established during the session and digitally signed with the negotiated post-quantum signature algorithm. This ensures both integrity and authenticity of the data source. The host verifies the signature and decrypts the ciphertext (data). In addition, LLM generates the final answer based on the validated result. Notably, this procedure remains fully compatible with the MCP interface, requiring no modifications to the existing client–server interaction model.

When the negotiated security level cannot be satisfied by the server’s supported algorithms, the response is still provided but accompanied by a reduced reliability score. This score allows users to gauge the degree of trust they should place in the LLM’s output, thereby supporting more transparent and trustworthy AI-assisted decision making. Furthermore, the underlying rationale for the

assigned score is logged, offering operational visibility and enabling follow-up actions where necessary.

3.5 Key Difference from Standard MCP

Table 2 highlights the security distinctions between the standard MCP and CA-MCPQ. Whereas MCP provides only optional or implementation-specific controls and omits critical safeguards, CA-MCPQ introduces protocol-level guarantees across all essential aspects, thereby transforming MCP into a secure-by-design architecture.

The proposed determination process further couples the agent’s contextual reasoning with post-quantum protocol negotiation, enabling natural-language task descriptions to be translated into enforceable cryptographic requirements.

Table 2: Key differences between standard MCP and CA-MCPQ.

Aspect	Standard MCP	CA-MCPQ
Authentication	$\triangle$	$\checkmark$
Encryption	$\triangle$	$\checkmark$
Authorization	$\times$	$\checkmark$
Session Integrity	$\times$	$\checkmark$
Quantum Resistance	$\times$	$\checkmark$

$\checkmark$ denotes mandatory support.

$\triangle$ indicates optional or implementation-specific support.

$\times$ denotes lack of support in the standard.

4 Evaluation

4.1 Experimental Setup

Table 3 shows the experimental environment. For cryptography, we use the CLOQS library from the PQC migration platform jointly provided by NIA and LG Uplus. This allows a single interface to both NIST standard PQC and Korean KpqC families, and it handles session key agreement and message authentication using PQC algorithms. Our implementation supports 13 KEMs (e.g., Kyber, SMAUG, and NTRU) and 27 signature algorithms (e.g., Dilithium and SPHINCS+).

In this experiment, two approaches to utilizing the AI model are considered: one involves downloading and executing the deployed model locally (i.e., local run), while the other entails accessing the production model via Claude’s API (i.e., API run).

Table 3: Summary of our experimental setup.

Operating System	Ubuntu 22.04
GPU	RTX4080
CPU	13th Gen Intel® Core™ i7-13700KF
RAM	16GB (GPU), 32GB (CPU)
LLM	Anthropic Claude, Google Gemma3, LG EXAONE
PQC Library	CLOQS library

Table 4: Summary of execution time (units: s).

LLM Type	Local	API
<i>Overall response time</i>		
Total time	9.0524	1.6648
<i>Stage-wise average time</i>		
Decision stage*	3.6945	0.6743
Security level decision	0.0001	0.0001
Algorithm selection	0.0380	0.0383
KEM encapsulation	0.0015	0.0007
Session establishment	0.0363	0.0371
Encryption	0.0008	0.0003
Network call	0.0398	0.0391
Decryption	0.0018	0.0017
Final answer generation	5.2394	0.8731

* In this phase, LLM determines the required security level.

4.2 Execution Time

Table 4 shows the response latency, the end-to-end time the AI takes to produce an answer. The API run is faster than the local run (1.6648s vs. 9.0524s), and this gap is caused by language model processing rather than protocol or our implementation. On the local run, model related stages (LLM Decision and Final Answer Generation) take 8.9339s out of 9.0524s ($\approx 98.7\%$). On the API run, the corresponding stages take 1.5474s out of 1.6648s ($\approx 93.0\%$). Other stages algorithm negotiation, KEM encapsulation, session establishment, AES-GCM encryption and decryption, and message routing are negligible runtime costs. On the API run, the combined cryptographic overhead is about 0.04s (roughly 2-3% of the total). These results are based on Kyber768 and AIMER192F. Although different PQC schemes may yield slightly different figures, the differences are not significant. Taken together, the findings mitigate common concerns that PQC integration necessarily degrades performance, and demonstrate that protocol-level

hardening can be realized without materially degrading user-perceived responsiveness.

4.3 Experimental Validation

We empirically validate that CA-MCPQ mitigates the principal attack vectors identified in Section 2.3.

Impersonation. Mutual post-quantum authentication—server signatures and client PQ challenge-response—is required to establish a session; any attempt without cryptographic proof of identity is blocked.

Token-based attack. We eliminate tokens at the protocol level: the client is never authenticated via pass-through access tokens. Consequently, impersonation vectors that rely on token passthrough or exfiltrated OAuth credentials are inapplicable: a token is neither accepted as a primary authenticator nor sufficient to complete the handshake or derive the session keys.

Session hijacking and replay attacks. Sequence numbers are bound into the AEAD additional authenticated data, so context and ordering are verified at decryption time and replayed or reused packets are rejected.

Over-privileged access. Permissions are enforced via RBAC after cryptographic validation; calls that fail authorization receive consistent encrypted denial responses, preventing unauthorized tool access.

Quantum cryptanalysis. PQC KEMs and post-quantum signatures are applied to session establishment and message signing, reducing the viability of future quantum-era decryption of recorded traffic.

Table 5: Summary of provided security features against attacks in MCP.

Provided Feature	Attacks
PQ mutual authentication	Impersonation, Token-based attacks
Session key binding	Session hijacking
Authenticated sequencing	Replay attacks, session reuse
RBAC	Over-privileged access
PQC (KEM & Signature)	Quantum cryptanalysis

5 Conclusion

In this work, we proposed CA-MCPQ, a context-aware post-quantum-secure extension of the Model Context Protocol. CA-MCPQ integrates contextual inference with protocol-level negotiation to select appropriate PQC algorithms, derives session keys through KEM, and enforces request integrity via authenticated sequencing and RBAC. A reliability score further guides agents in heterogeneous environments by quantifying how well server capabilities align with required security levels. Beyond these mechanisms, CA-MCPQ introduces protocol-level assurances for authentication, encryption, authorization, and quantum resistance, while preserving full compatibility with existing MCP workflows.

Our evaluation shows that the cryptographic and transport overheads are negligible compared to model computation, demonstrating that CA-MCPQ achieves strong cryptographic assurances and crypto agility without sacrificing performance or interoperability. As a result, CA-MCPQ provides a practical path toward secure-by-design agent ecosystems capable of resisting both current and emerging quantum threats.

Future work will focus on enhancing protocol efficiency and broadening security coverage. In particular, we aim to reduce latency through further optimization of cryptographic handshakes and session management, while exploring mechanisms that maintain strong post-quantum assurances. We also plan to extend the threat model to address additional attack surfaces, thereby strengthening the robustness of CA-MCPQ in practical deployments. Finally, we will investigate extensions toward agent-agent communication protocols, enabling end-to-end secure coordination across distributed AI ecosystems.

References

1. NIST., “Submission requirements and evaluation criteria for the post-quantum cryptography standardization process,” 2016. <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>. 2
2. NIST., “Call for additional digital signature schemes for the post-quantum cryptography standardization process,” 2022. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf>. 2
3. IBM, “What are ai agents?.” <https://www.ibm.com/think/topics/ai-agents>, 2025. Accessed: 2025-09-16. 3
4. Google Cloud, “What are ai agents? definition, examples, and types.” <https://cloud.google.com/discover/what-are-ai-agents>, 2025. Accessed: 2025-09-16. 3
5. Model Context Protocol Working Group, “Mcp tools, schemas, and meta-data.” <https://modelcontextprotocol.io/docs/concepts/tools>, 2025. Accessed: 2025-09-16. 4
6. Model Context Protocol Working Group, “Mcp architecture overview.” <https://modelcontextprotocol.io/docs/concepts/architecture>, 2025. Accessed: 2025-09-16. 4

7. R. Hat, “Model context protocol (mcp): Understanding security risks and controls.” <https://www.redhat.com/en/blog/model-context-protocol-mcp-understanding-security-risks-and-controls>, 2024. Accessed: 2025-09-16. 5, 6
8. F. C. Gabarda, “Model context protocol (mcp): Understanding security risks and controls.” <https://www.redhat.com/en/blog/model-context-protocol-mcp-understanding-security-risks-and-controls>, 2025. Red Hat Blog, Accessed: 2025-09-16. 5, 6, 7
9. Model Context Protocol Working Group, “Mcp security best practices (draft specification).” https://modelcontextprotocol.io/specification/draft/basic/security_best_practices, 2025. Accessed: 2025-09-16. 5, 6, 7
10. D. Sarig, “The security risks of model context protocol (mcp).” <https://www.pillar.security/blog/the-security-risks-of-model-context-protocol-mcp>, 2025. Accessed: 2025-09-16. 5
11. P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings 35th annual symposium on foundations of computer science*, pp. 124–134, Ieee, 1994. 6
12. L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. Perlner, and D. Smith-Tone, “NIST released NISTIR 8105 report on post-quantum cryptography,” *Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep.*, 2016. 6
13. L. K. Grover, “A fast quantum mechanical algorithm for database search,” in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996. 6
14. W. Castryck and T. Decru, “An efficient key recovery attack on sidh,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 423–447, Springer, 2023. 6